

CYBER SECURITY LAB EXPERIMENT

Title: Privacy Audit of Popular Apps (Desktop **WhatsApp**) and Websites (**Facebook**) & Data Breach Case Study Analysis

Aim

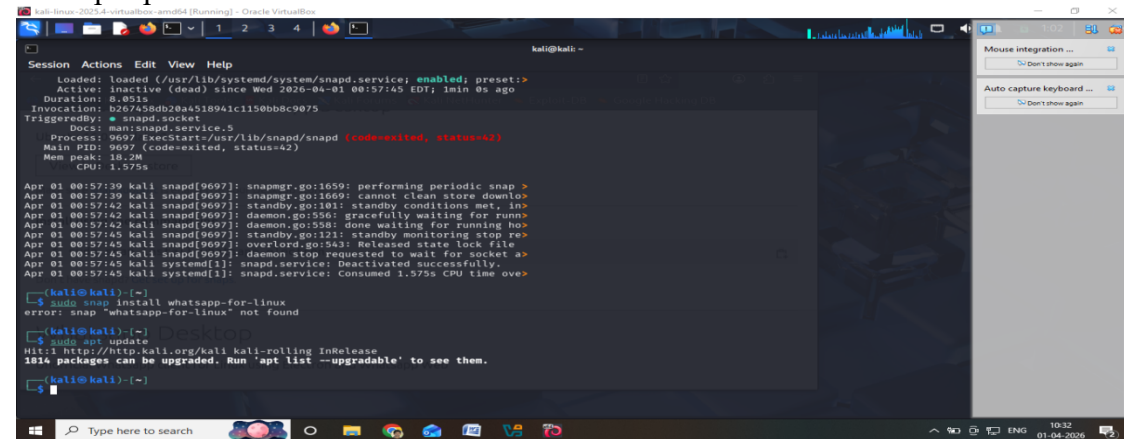
To analyze privacy, network behavior, and security risks of applications and websites using Kali Linux tools.

Tools Required

- **Wireshark**
- **Burp Suite**
- Kali Linux (VirtualBox)
- Desktop WhatsApp (Nativefier)
- Web Browser (Firefox)

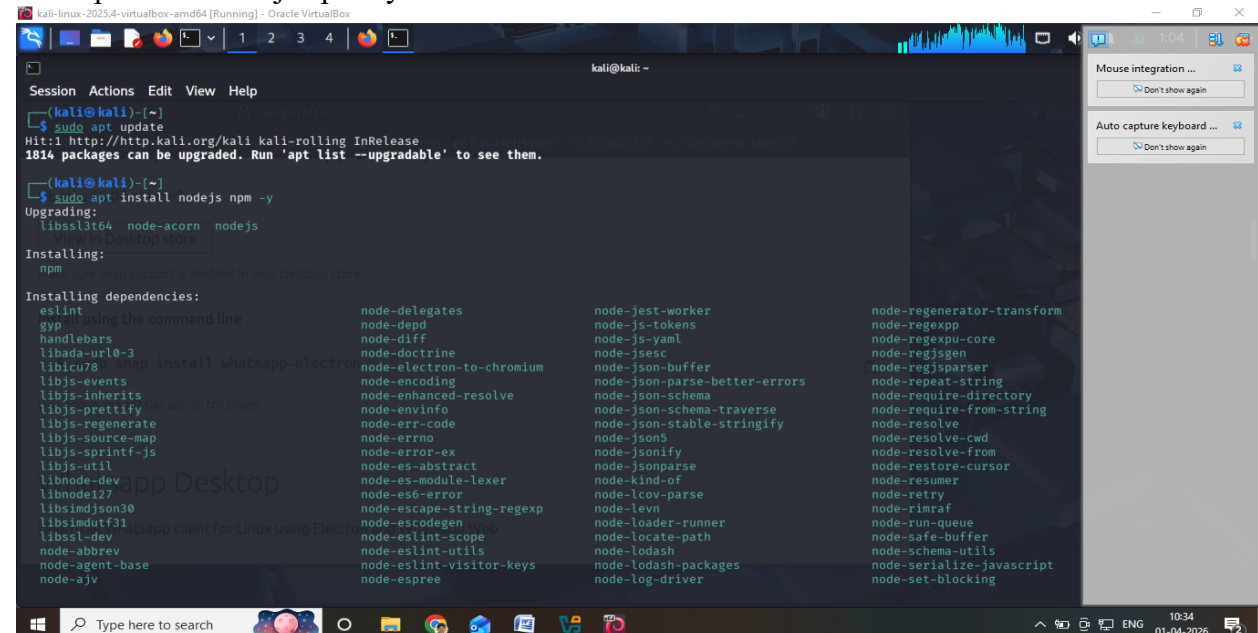
Step 1: Install Node.js & npm

sudo apt update

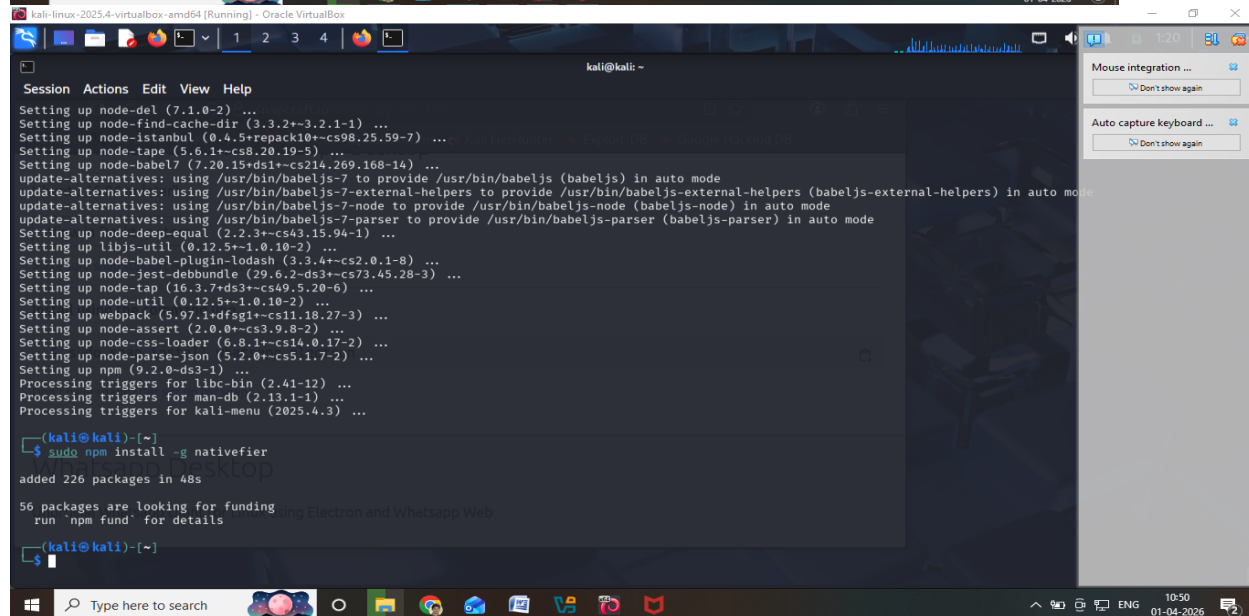


```
kali@kali: ~  
$ sudo apt update  
Hit:1 http://http.kali.org/kali kali-rolling InRelease  
1814 packages can be upgraded. Run 'apt list --upgradable' to see them.  
$
```

sudo apt install nodejs npm -y

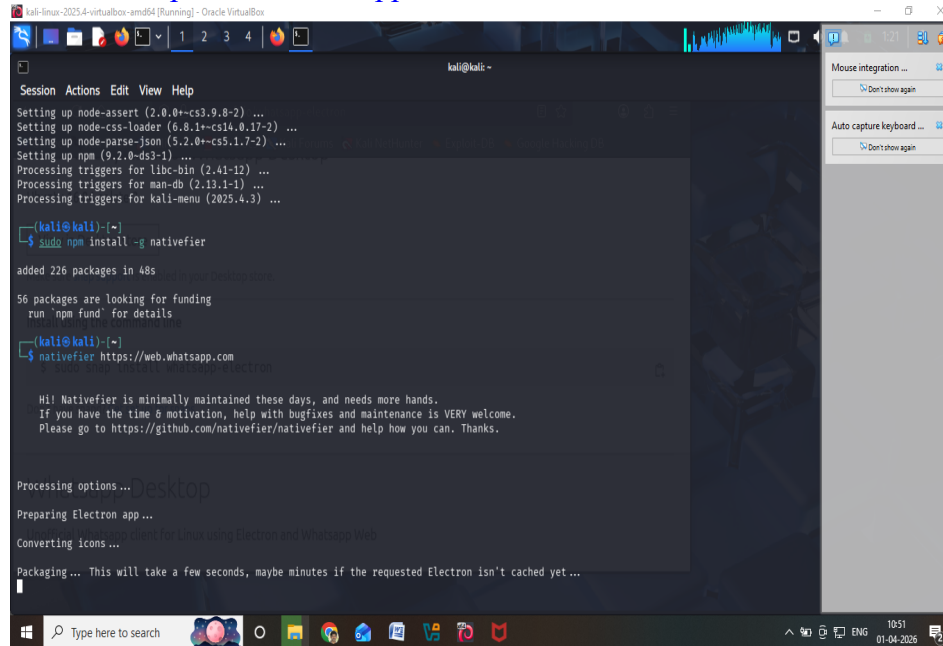


```
kali@kali: ~  
$ sudo apt install nodejs npm -y  
Upgrading:  
  libssl3t64 node-acorn nodejs  
Installing:  
  npm  
Installing dependencies:  
  eslint gyp handlebars libbada-url0-3 libicu78 libjs-events libjs-inherits libjs-prettify libjs-regenerate libjs-source-map libjs-sprintf-js libjs-util libnode-dev libnode127 libsimdjson30 libssmoutf32 libssl-dev node-abbrev node-agent-base node-a3v node-delegates node-depd node-diff node-doctrine node-electron-to-chromium node-encoding node-enhanced-resolve node-envinfo node-err-code node-errno node-error-ex node-es-abstract node-es-module-lexer node-es6-error node-escape-string-regexp node-escodegen node-eslint-scope node-eslint-utils node-eslint-visitor-keys node-espree node-jest-worker node-js-tokens node-js-yaml node-jsesc node-json-buffer node-json-parse-better-errors node-json-schema node-json-schema-traverse node-json-stable-stringify node-json5 node-jsonify node-jsonparse node-kind-of node-levn node-lcov-parse node-levn node-loader-runner node-locate-path node-lodash node-lodash-packages node-log-driver node-regenerator-transform node-regexpp node-regexu-core node-regjsen node-regjsparser node-repeat-string node-require-directory node-require-from-string node-resolve node-resolve-cwd node-resolve-from node-restore-cursor node-resumer node-retry node-rimraf node-run-queue node-safe-buffer node-schema-utils node-serialize-javascript node-set-blocking
```



Step 3: Create WhatsApp Desktop App

nativefier <https://web.whatsapp.com>



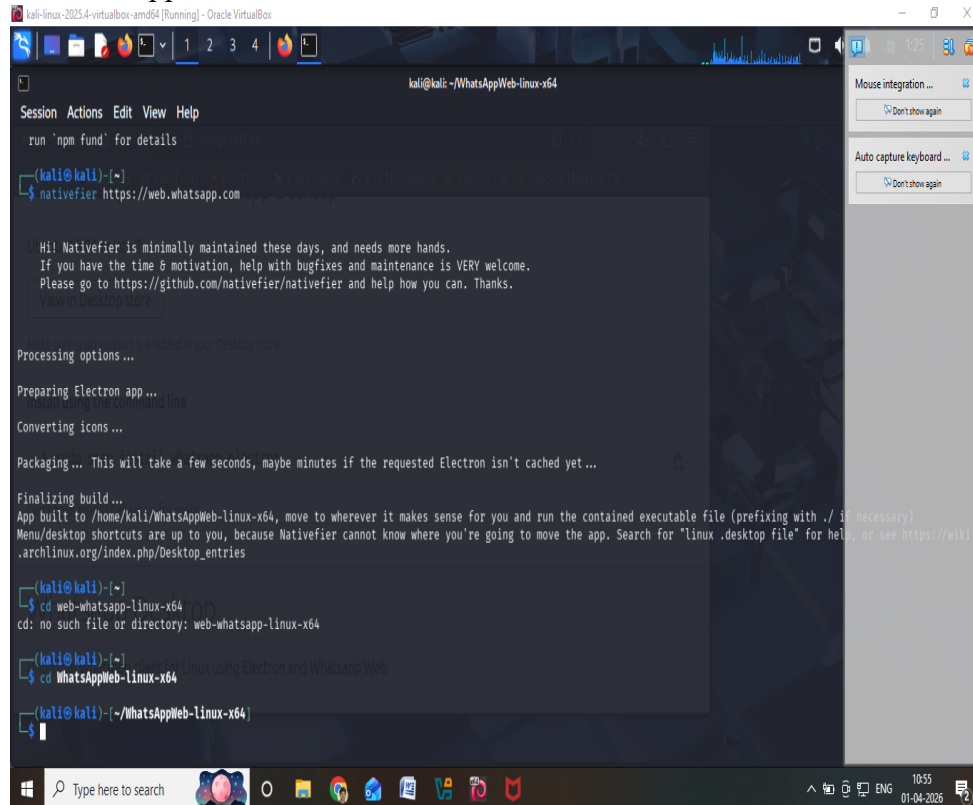
```
kali@kali: ~$ sudo npm install -g nativefier
added 226 packages in 48s
56 packages are looking for funding
  run `npm fund` for details

(kali@kali)~$ nativefier https://web.whatsapp.com
Processing options... Desktop
Preparing Electron app...
Converting icons...
Packaging... This will take a few seconds, maybe minutes if the requested Electron isn't cached yet...
```

It will create a folder like:
web-whatsapp-linux-x64

Step 4: Run WhatsApp

cd WhatsAppWeb-linux-x64



```
kali@kali: ~/WhatsAppWeb-linux-x64
run `npm fund` for details

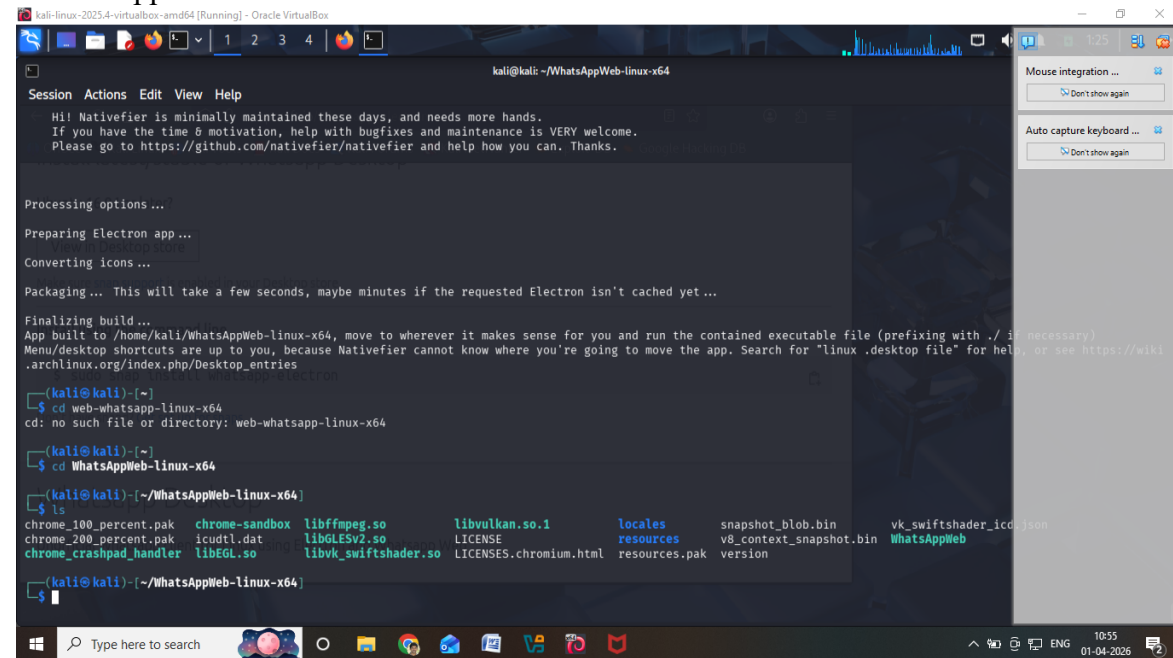
(kali@kali)~$ nativefier https://web.whatsapp.com
Processing options... Desktop
Preparing Electron app...
Converting icons...
Packaging... This will take a few seconds, maybe minutes if the requested Electron isn't cached yet...

Finalizing build...
App built to /home/kali/WhatsAppWeb-linux-x64, move to wherever it makes sense for you and run the contained executable file (prefixing with ./ if necessary)
Menu/desktop shortcuts are up to you, because Nativefier cannot know where you're going to move the app. Search for "linux .desktop file" for help, or see https://wiki.archlinux.org/index.php/Desktop_entries

(kali@kali)~$ cd web-whatsapp-linux-x64
cd: no such file or directory: web-whatsapp-linux-x64

(kali@kali)~$ cd WhatsAppWeb-linux-x64
(kali@kali)~$ cd ~/WhatsAppWeb-linux-x64
```

./WhatsAppWeb



```
kali@kali: ~/WhatsAppWeb-linux-x64
Session Actions Edit View Help
Hi! Nativefier is minimally maintained these days, and needs more hands.
If you have the time & motivation, help with bugfixes and maintenance is VERY welcome.
Please go to https://github.com/nativefier/nativefier and help how you can. Thanks.

Processing options...
Preparing Electron app...
Converting icons...
Packaging... This will take a few seconds, maybe minutes if the requested Electron isn't cached yet...
Finalizing build...
App built to /home/kali/WhatsAppWeb-linux-x64, move to wherever it makes sense for you and run the contained executable file (prefixing with ./ if necessary)
Menu/desktop shortcuts are up to you, because Nativefier cannot know where you're going to move the app. Search for "linux .desktop file" for help, or see https://wiki.archlinux.org/index.php/Desktop_entries

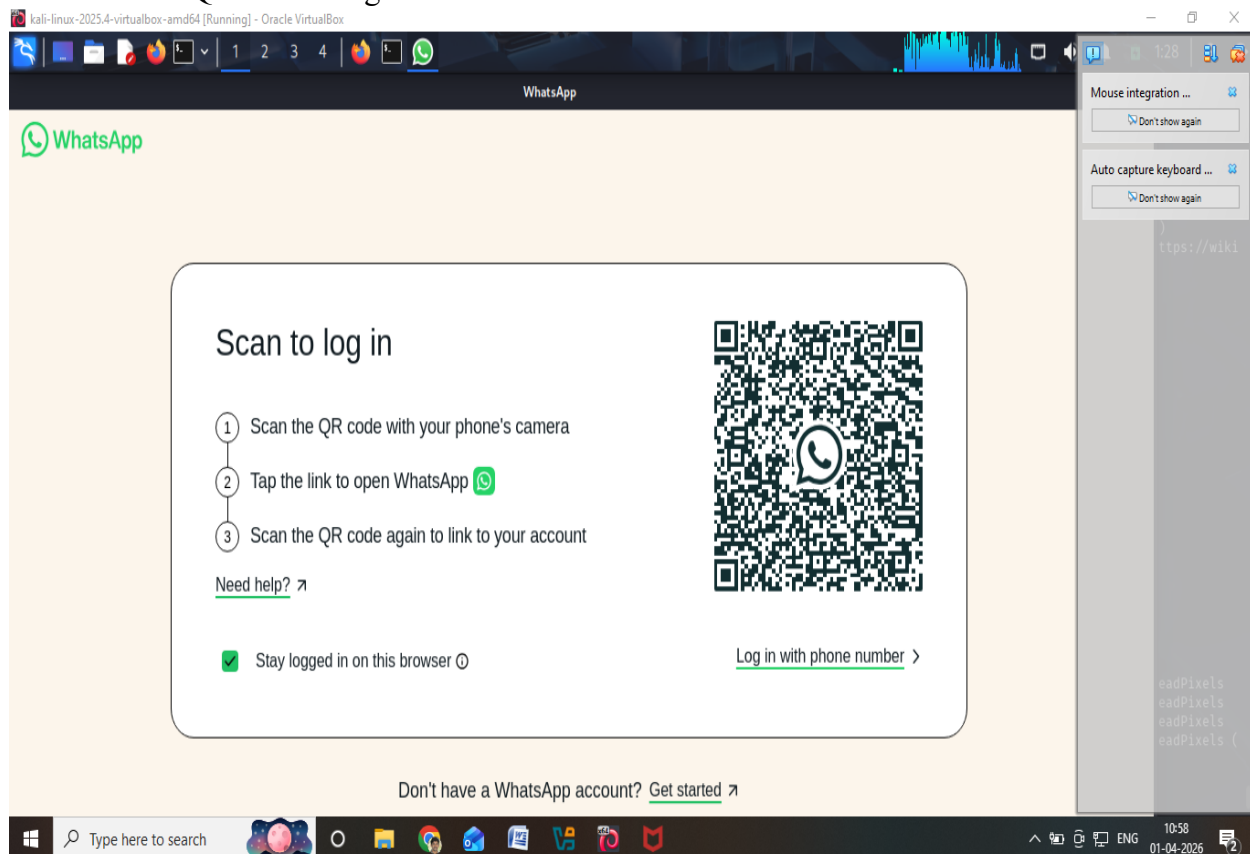
(kali@kali)~$ cd web-whatsapp-linux-x64
cd: no such file or directory: web-whatsapp-linux-x64

(kali@kali)~$ cd WhatsAppWeb-linux-x64

(kali@kali)~/WhatsAppWeb-linux-x64$ ls
chrome_100_percent.pak  chrome-sandbox  libffmpeg.so  libvulkan.so.1  locales  snapshot_blob.bin  vk_swiftshader_icd.json
chrome_200_percent.pak  icudtl.dat      libGLESv2.so  LICENSE         resources  v8_context_snapshot.bin  WhatsAppWeb
chrome_crashpad_handler  libEGL.so       libvk_swiftshader.so  LICENSES.chromium.html  resources.pak  version
```

✓ Step 5: Login

- Scan QR code using mobile



- Done

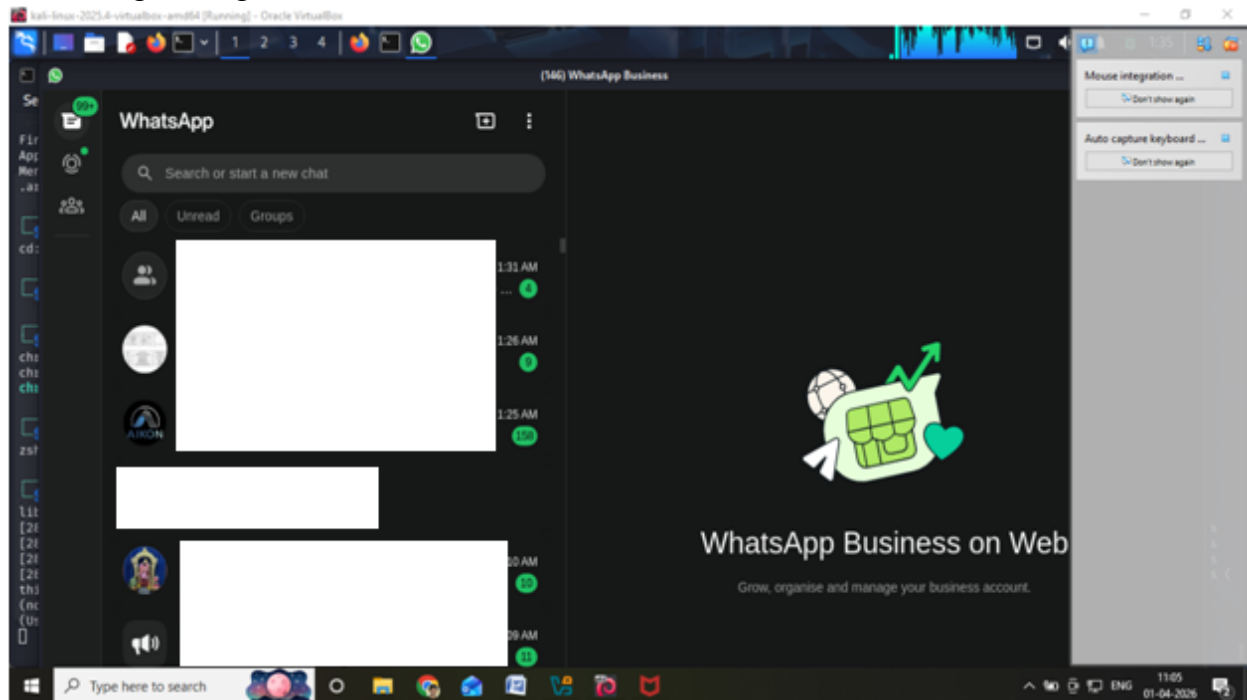
PART 1: Privacy Audit of Desktop WhatsApp

Step 1: Launch WhatsApp Desktop

cd WhatsAppWeb-linux-x64

./WhatsAppWeb

- QR Code appears
- Login using mobile → Linked Devices

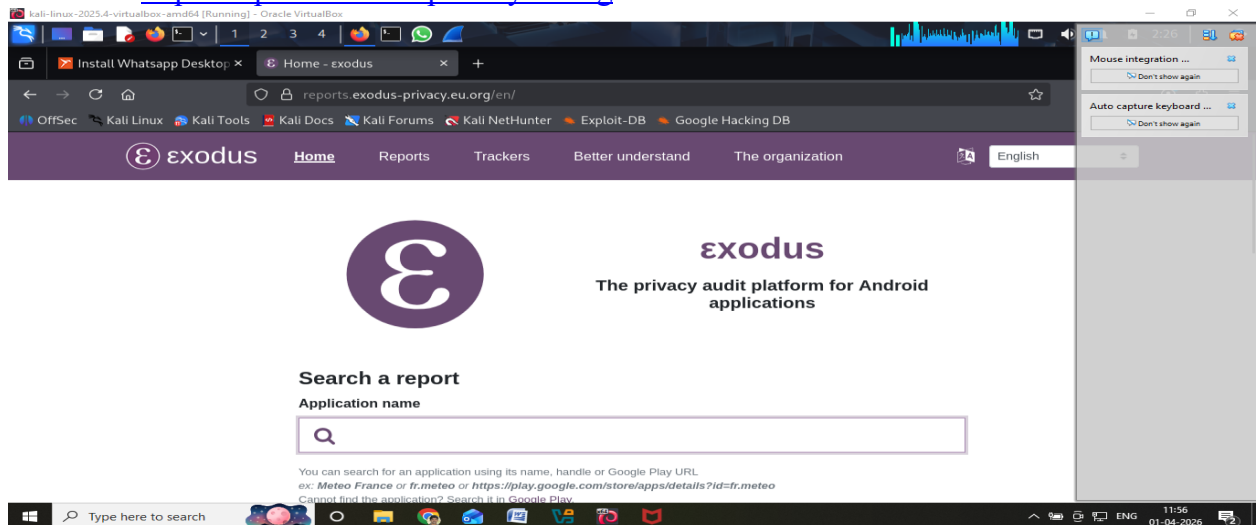


Step 1: Install WhatsApp APK (Optional in Emulator)

- Download APK from trusted source or use your mobile

Step 2: Analyze Trackers using Exodus Privacy

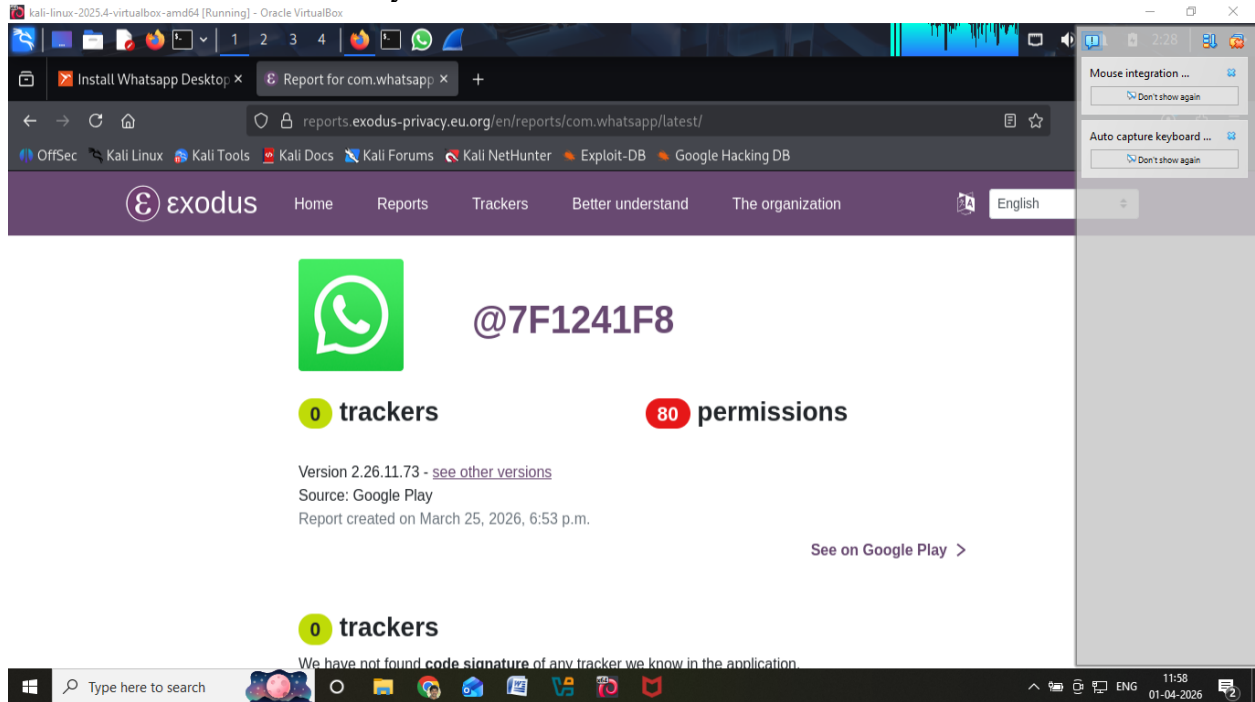
1. Open browser in Kali
2. Go to: <https://reports.exodus-privacy.eu.org>



3. Search for WhatsApp

Observation:

- Detects trackers like:
 - Google Analytics
 - Crashlytics
 - Facebook Analytics

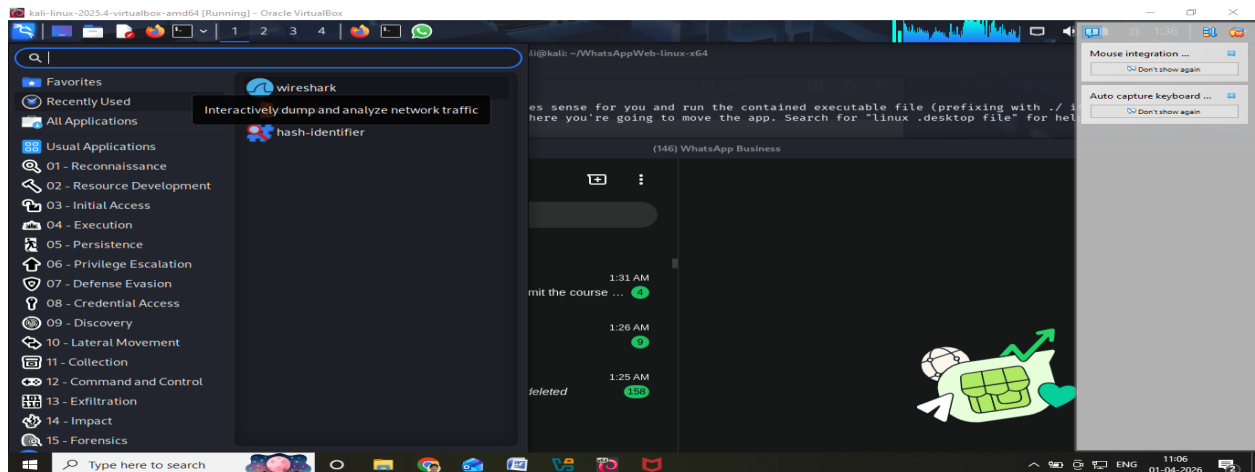


Record in lab:

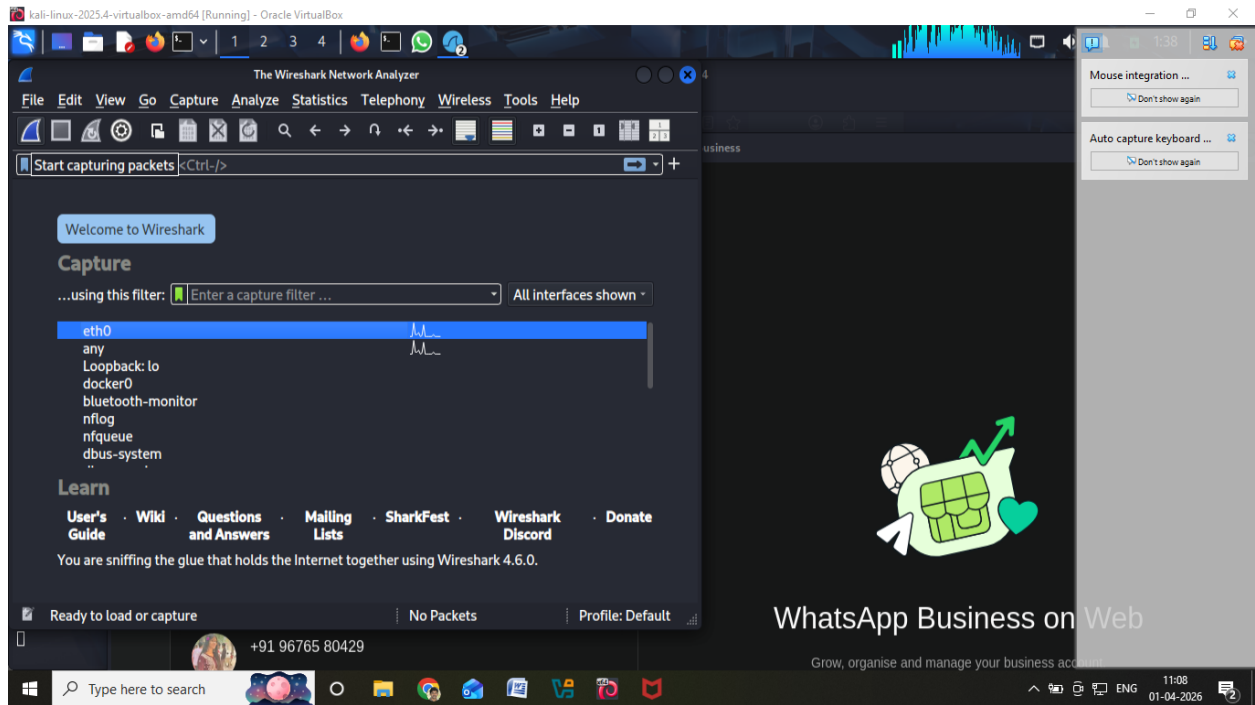
- Number of trackers
- Permissions used

Step 2: Start Wireshark

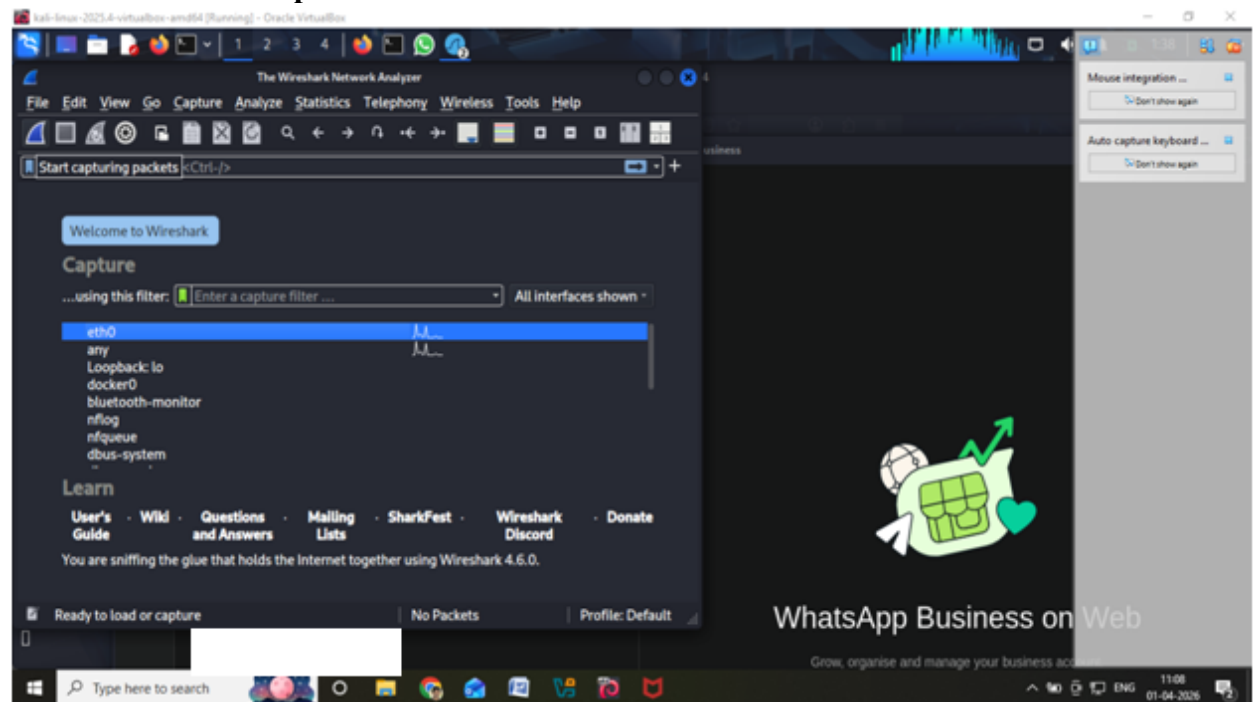
Wireshark



- Select interface (eth0 / wlan0)



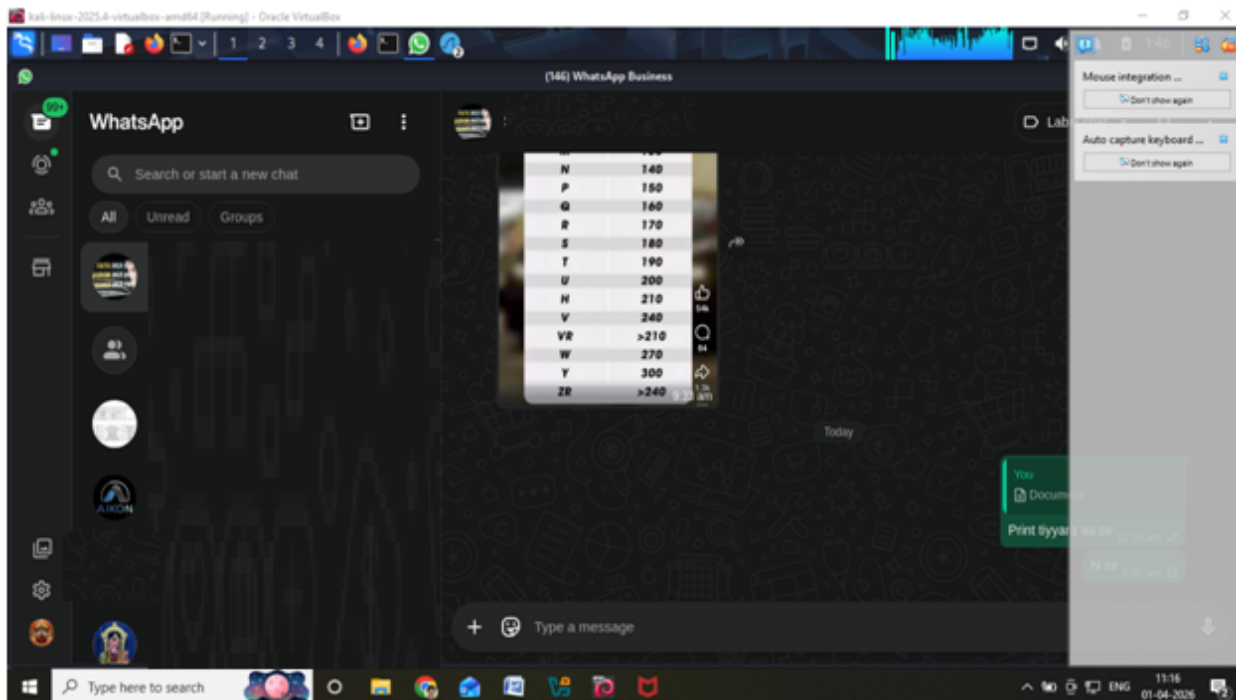
-
- Click Start Capture



Step 3: Generate Traffic

In WhatsApp Desktop:

- Send messages
- Send images
- Open chats

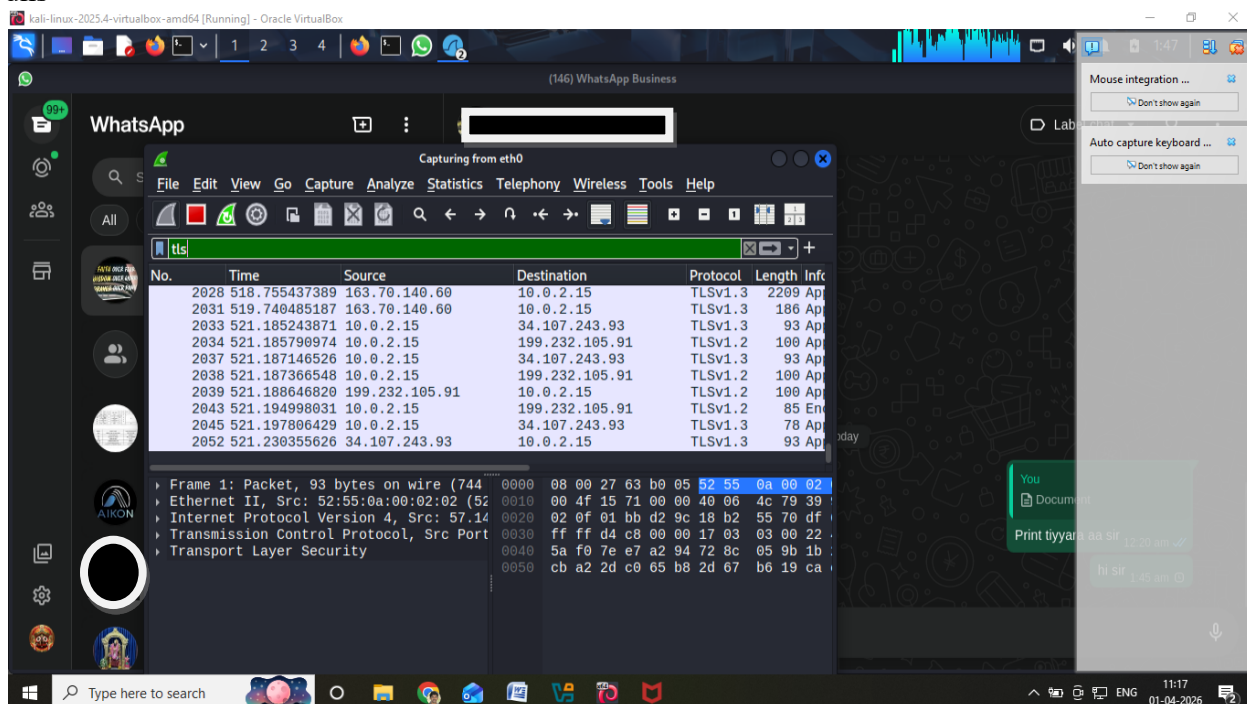


✓ Step 4: Apply Filters

Use filters:

tls

dns

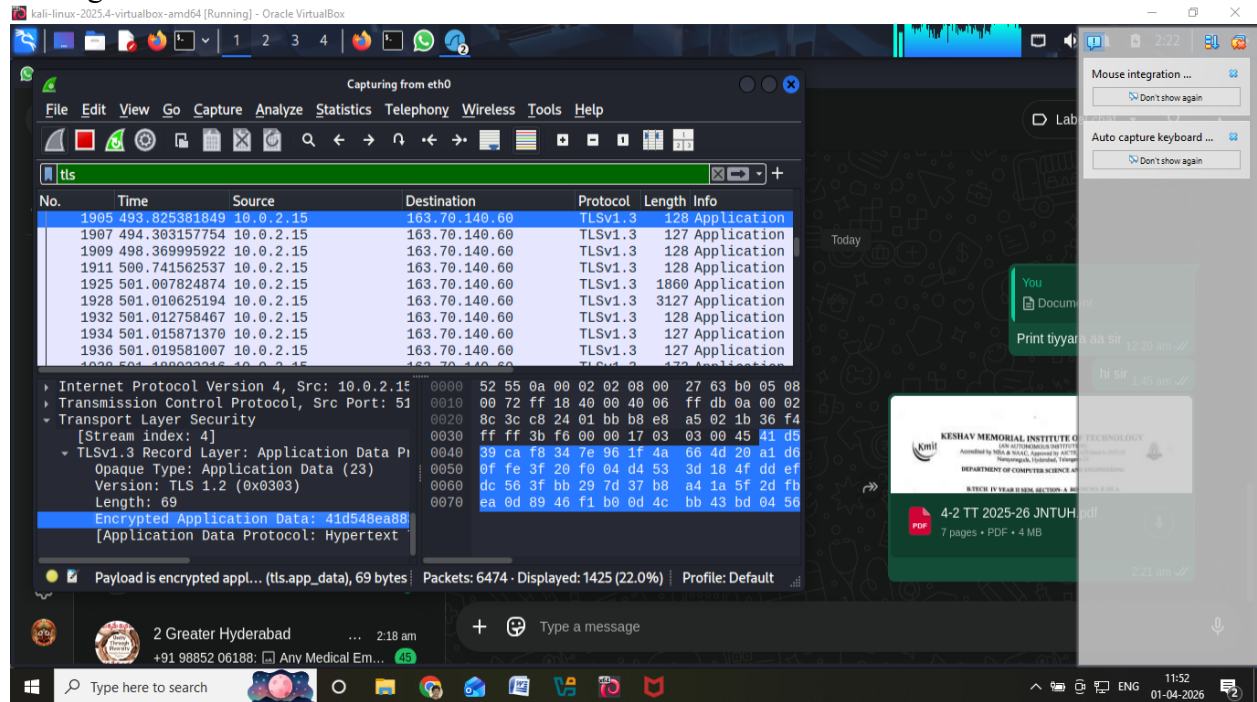


Observations

- Encrypted traffic (TLS packets)
- Domains:

- whatsapp.net
- facebook.com

Message content NOT visible



Findings

End-to-End Encryption

Secure communication

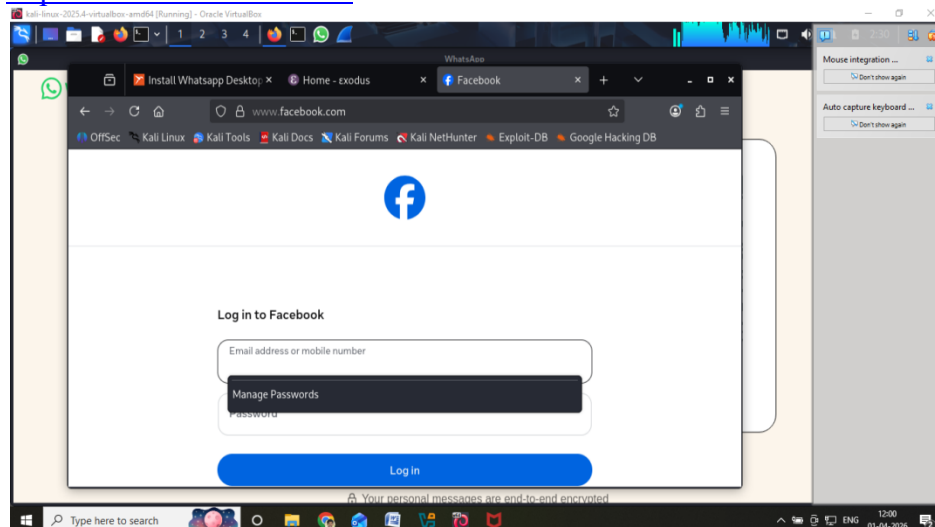
Metadata visible:

- IP address
- Time
- Server communication

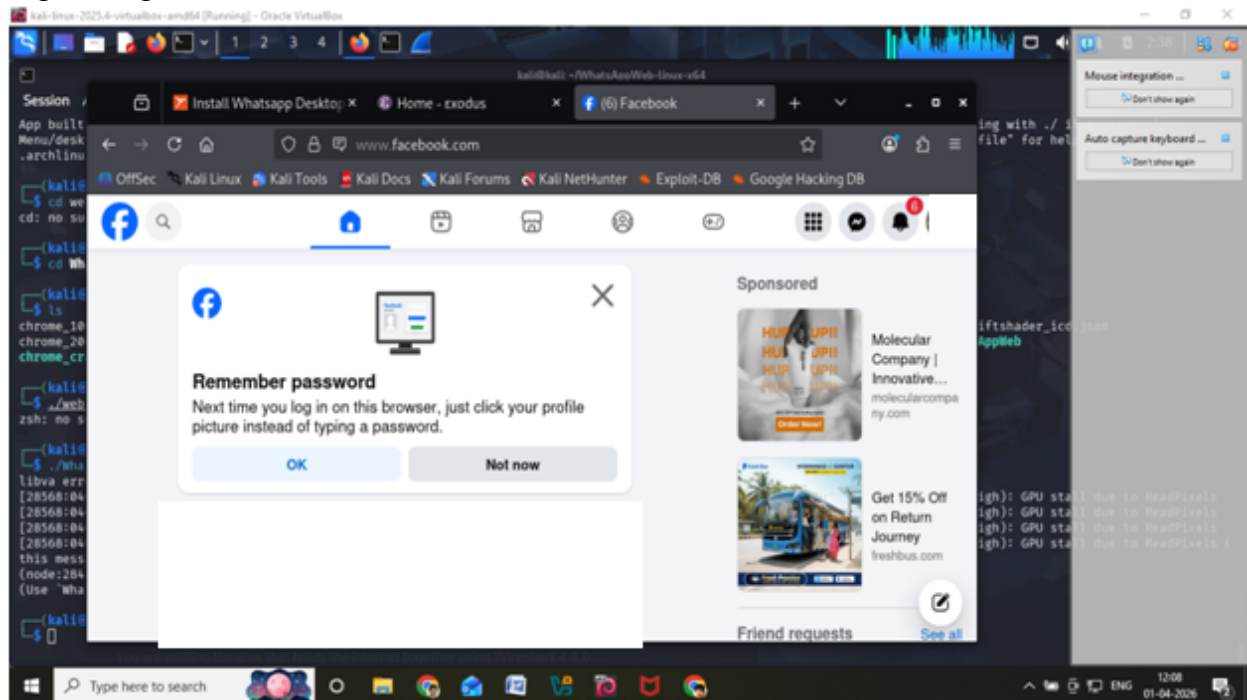
PART 2: Privacy Audit of Facebook Website

Step 1: Open Facebook

<https://www.facebook.com>

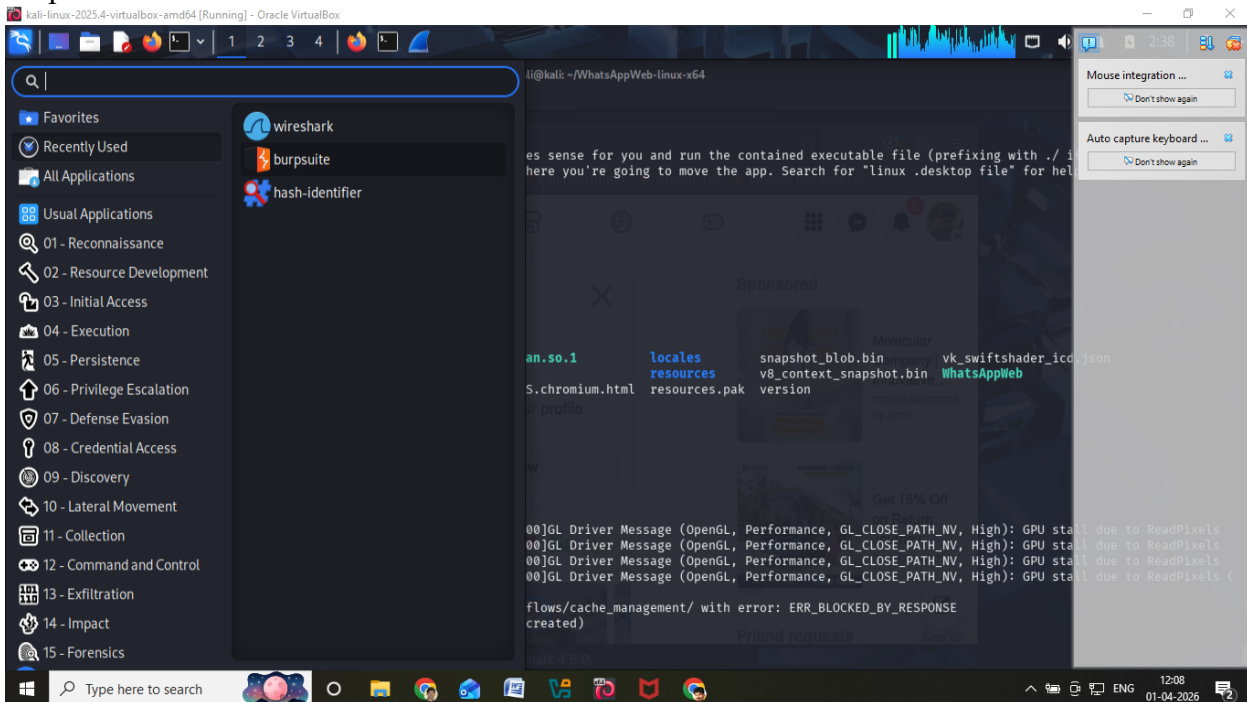


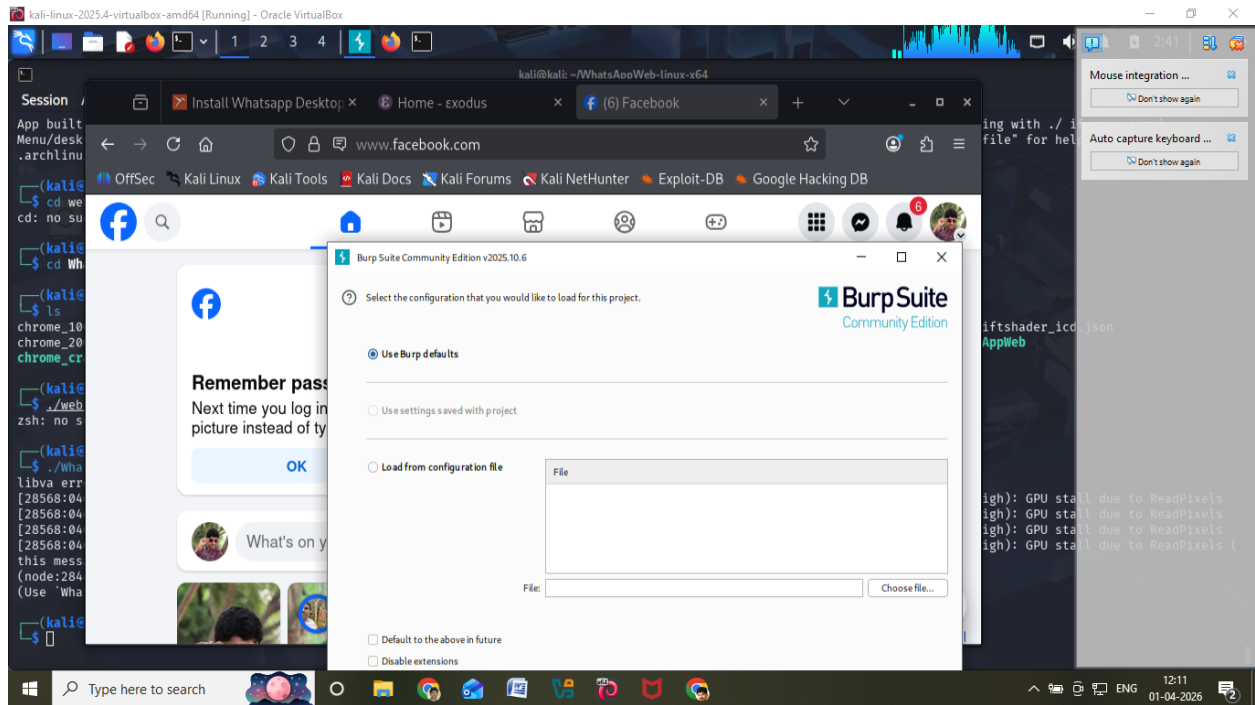
Login using test account



Step 2: Start Burp Suite

Burpsuite



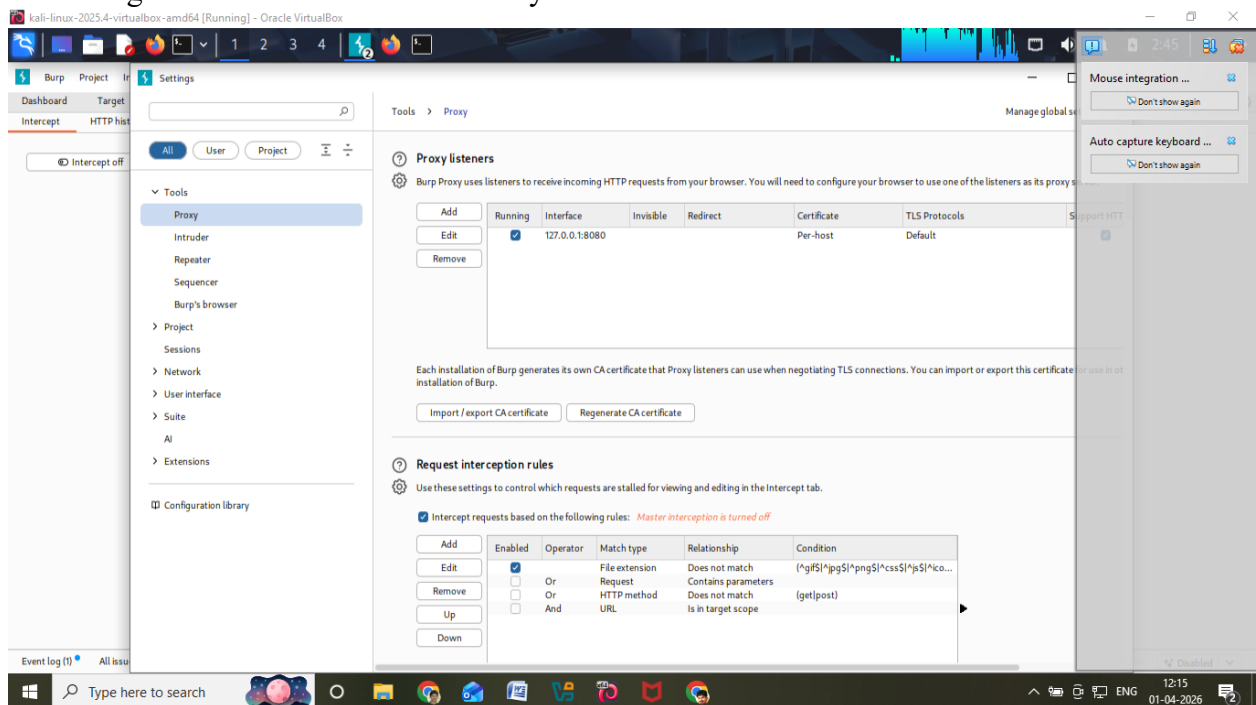


✓ Step 3: Configure Proxy

- IP: 127.0.0.1
- Port: 8080

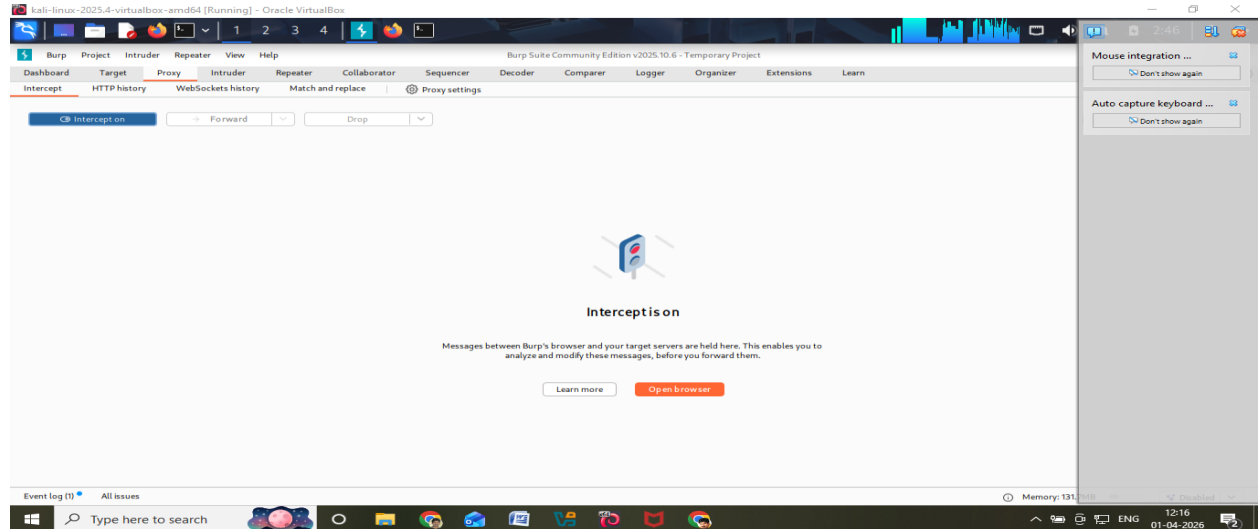
In browser:

- Settings → Network → Manual Proxy



✓ Step 4: Intercept Traffic

- Turn **Intercept ON**
- Reload Facebook



Observe:

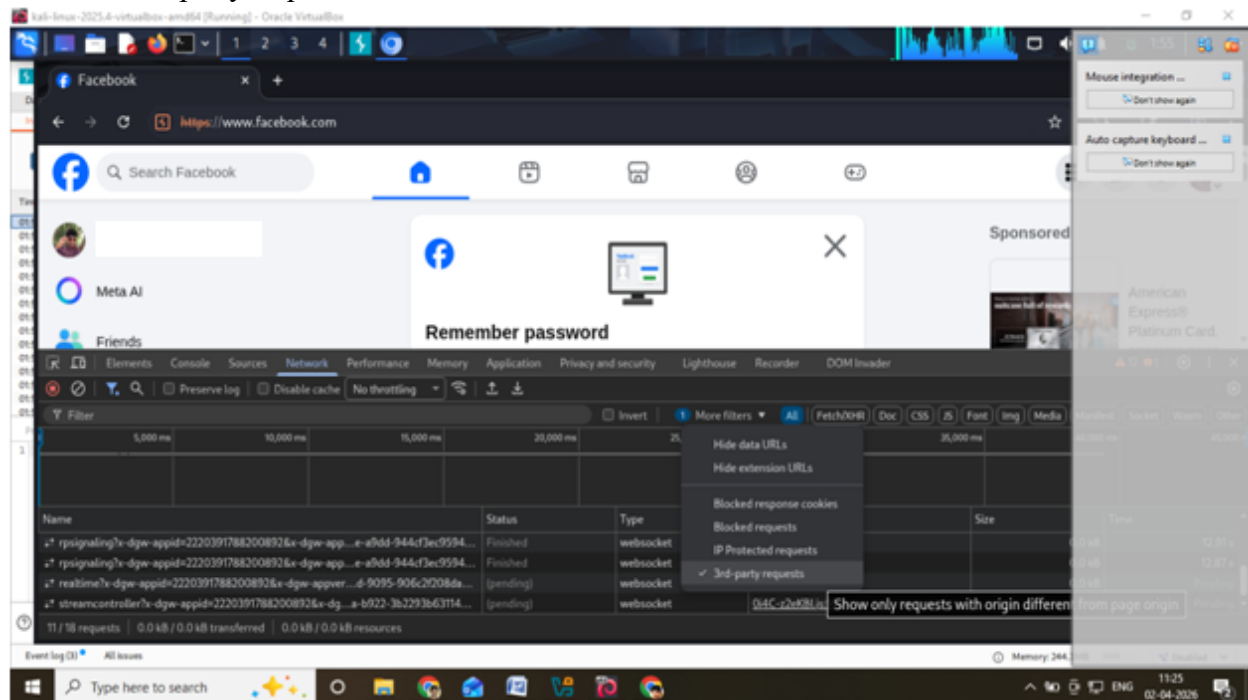
- Cookies (datr, fr)
- Session IDs
- Headers

Step 5: Analyze Tracking

Press **F12** → **Network** tab

Look for:

- Ads
- Analytics scripts
- Third-party requests



Findings

Heavy tracking

Uses cookies for personalization

Third-party integrations

PART 3: Data Breach Case Study

Case Study 1: Facebook Data Breach

- 533 million users affected
- Data leaked:
 - Phone numbers
 - Emails

Cause:

- API vulnerability

Case Study 2: WhatsApp Pegasus Attack

- Spyware attack via missed call
- Remote device access

Step 3: Check Email Breach

Go to:

👉 <https://haveibeenpwned.com>

Enter email → check breach status

Analysis Points

- Type of attack
- Vulnerability
- Data exposed
- Prevention

Observations

- WhatsApp uses strong encryption
- Facebook performs extensive tracking
- Data breaches occur due to vulnerabilities

Result

Privacy audit was successfully performed using Kali Linux tools. WhatsApp ensures secure communication through encryption, while Facebook shows extensive tracking behavior. Data breach analysis highlighted risks of poor security practices.